計算幾何

如何用精確的幾何判定，設計有效率且正確的演算法？

本章依序學習 **方向判定 → 線段相交與掃描線 → 凸包 → 最近點對**。前置知識包括排序、平衡搜尋樹、分治法與漸近分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從幾何圖形到演算法

問題	學習重點
方向與線段相交	行列式符號、邊界與數值精度。
掃描線	事件順序、狀態與相鄰性。
凸包	排序、堆疊不變量與退化情況。
最近點對	分治、帶狀區域與幾何打包。

每個結果都依賴明確的輸入假設與正確性證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用局部符號回答幾何問題

本節以平面點與閉線段為輸入。先固定座標與相交語義，再以方向判定組成線段相交測試，最後用掃描線回答「是否存在一對相交線段」。

所有數值例使用精確整數或有理數。時間界先按每次精確算術為常數的模型計算；大整數位元成本另計。來源為 CLRS 33.1–33.2 與 MIT 2004 L17；原筆記 PDF43–47 提供幾何章脈絡與轉向圖。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左轉的正號取決於公式順序

固定 x 軸向右、y 軸向上。

$$\operatorname{orient}(a,b,c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

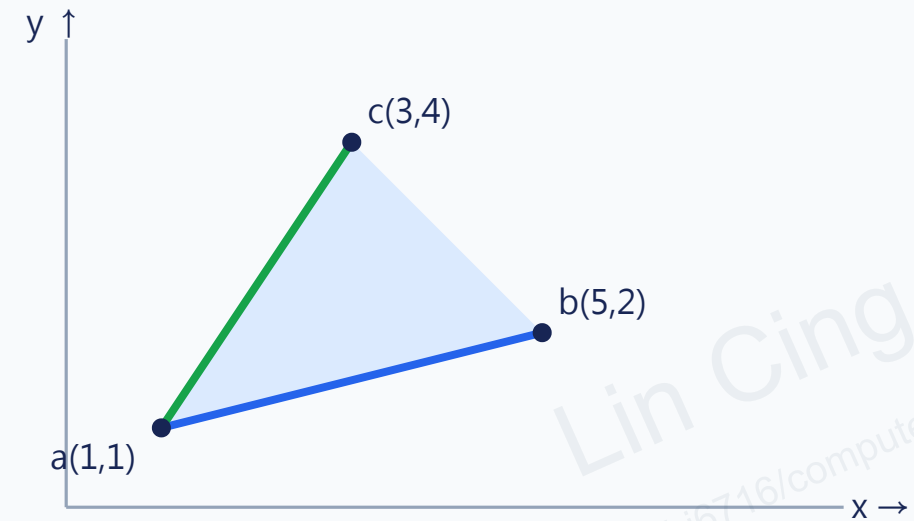
正值表示從 a 經 b 到 c 左轉，負值為右轉，零表示共線。絕對值是平行四邊形面積，三角形面積為其一半。

CLRS 的 `DIRECTION(a,b,c)` 使用 $(c - a) \times (b - a)$，恰為本式負值。引用書中不等號時必須一起換號。SVG 畫面向下的 y 軸由繪圖轉換處理，不改變數學慣例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左轉的正號取決於公式順序 (續)

方向值的正負與座標



$(b-a) \times (c-a)$

$4 \cdot 3 - 1 \cdot 2 = +10$

左轉；三角形面積 5

DIRECTION = −10

y 軸向上

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

實算一次行列式

令 a=(1,1)、b=(5,2)、c=(3,4)。差向量 b−a=(4,1)，c−a=(2,3)。

$$\operatorname{orient}(a,b,c) = 4\cdot 3 - 1\cdot 2 = 10 > 0.$$

因此 c 在有向直線 ab 的左側，三角形面積為 5。交換 b,c 得 −10；平移三點不改變差向量。若 c=(9,3)，則差向量為 (8,2)，行列式 4・2−1・8=0，僅能推得共線，不能推得 c 在閉線段 ab 內。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

閉線段相交與嚴格交叉

閉線段 $ab = \{a + t(b - a) : 0 \le t \le 1\}$ 含端點，也允許 a=b 的零長度線段。

本節將「嚴格交叉」定義為兩個非共線線段的內部單點交會。若題目把共線內部重疊也稱為嚴格相交，必須另訂規格。

幾何情形	閉線段相交	嚴格交叉
兩內部穿越	是	是
端點接觸、T 形接觸	是	否
共線重疊	是	否
點線段落在另一段上	是	否
僅延長線相交	否	否

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

閉線段相交與嚴格交叉 （續）

閉線段邊界案例



嚴格交叉　端點接觸　共線重疊　點線段

藍、紅為輸入；綠為共同線段；金點為接觸點。

四例皆為閉段相交；只有第一例是嚴格交叉。

端點、共線與零長度必須由規格明確涵蓋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

on 必須同時檢查共線與兩軸界限

```
on(a,b,p):
    return orient(a,b,p) == 0 and
           min(ax,bx) <= px <= max(ax,bx) and
           min(ay,by) <= py <= max(ay,by)
```

a=(2,1)、b=(2,5)、p=(2,8) 雖共線且通過 x 界限，仍不在 ab 上，因此不能刪掉 y 界限。若 a=b，兩軸界限共同要求 p=a，故點線段也有定義。

CLRS 的 ON-SEGMENT 將「已知共線」交給呼叫端保證。本講義將它合併到 on，避免單用包圍盒就誤判。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四個方向值完成測試

記 o1=orient(a,b,c)、o2=orient(a,b,d)、o3=orient(c,d,a)、o4=orient(c,d,b)。

```
opposite(u,v): return (u<0 and v>0) or (u>0 and v<0)
intersect(a,b,c,d):
    compute o1,o2,o3,o4
    if opposite(o1,o2) and opposite(o3,o4): return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

第一個分支恰為嚴格交叉；第二分支處理端點、共線重疊與零長度。比較正負，不直接乘 o1·o2，避免再增加乘積溢位風險。垂直線段不必求斜率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

必須是雙向跨越

ab 為 (0,0)→(6,0)，cd 為 (7,−1)→(7,2)。

$$(o_1, o_2, o_3, o_4) = (-6, 12, 21, 3).$$

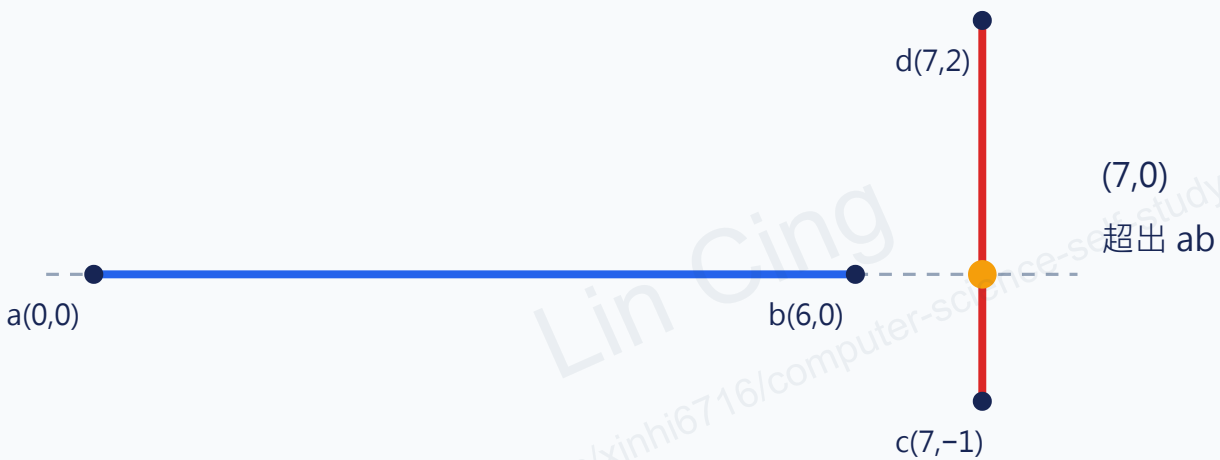
c,d 跨過 ab 所在直線，但 a,b 同在 cd 所在直線一側。交點 (7,0) 超出 ab，故閉線段不相交。

一般位置下，第一組異號使直線交點落在 cd 內，第二組異號使同一交點也落在 ab 內。兩組條件缺一不可。共線時，兩個閉區間若重疊，必有某個端點落在另一區間內，因而四個 on 測試足夠。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

必須是雙向跨越 （續）

跨越延長線仍可能不相交



o1=−6, o2=12；o3=21, o4=3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

精確符號需要足夠位元

若每個座標絕對值至多 B，差值至多 2B，保守界為 $|\,\mathrm{orient}\,| \le 8B^2$。先將座標擴寬，再做減法與乘法；計算完成才轉型已太晚。

a=(0,0)、b=(100000,0)、c=(0,100000) 的方向值為 10^{10}，已超出有號 32 位元範圍。完整有號 32 位元座標也不保證有號 64 位元中間值安全。

浮點相減可能消去有效位數。縮放座標 λ 倍會令行列式乘 λ^2，固定 epsilon 因此不具尺度無關保證。將「差距小於 epsilon」當排序相等，還可能破壞傳遞性。需要精確算術，或有誤差界的過濾器加精度提升；本節不把任意 epsilon 當證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從所有配對轉成事件

n 段逐對測試花 $\Theta(n^2)$ 次判定。存在性問題只需找到一對；列舉問題須輸出全部 k 對。k 可達 $\Theta(n^2)$，故不能把存在性 $O(n \log n)$ 誤寫為列舉所有交點的時間。

掃描線以 x 當時間。事件佇列保存線段端點；狀態 T 保存目前穿過掃描線的線段，以下到上排列。

先講清簡化版本的前提。每段非垂直、非零長度，端點 x 互異，且無三段同點相交，也無共線重疊。閉線段測試本身沒有這些限制；這些是掃描狀態與證明的前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

插入與刪除都可能產生新鄰居

```
T = empty balanced search tree
sort the 2n endpoints by x
for endpoint p:
    if p is the left endpoint of s:
        insert s in T
        test s against predecessor and successor; if any intersects: return true
    else:
        test predecessor(s) against successor(s); if they intersect: return true
        delete s from T
return false
```

測試的是完整線段是否相交，不只測試當下掃描線位置。故可以在實際交點之前回傳。刪除時必須測原本隔著 s 的兩段，否則會漏掉新相鄰配對。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用座標逐事件追蹤

a:(0,0)→(10,5)，c:(1,2)→(3,2)，b:(2,6)→(9,0)。下到上的 T 如下。

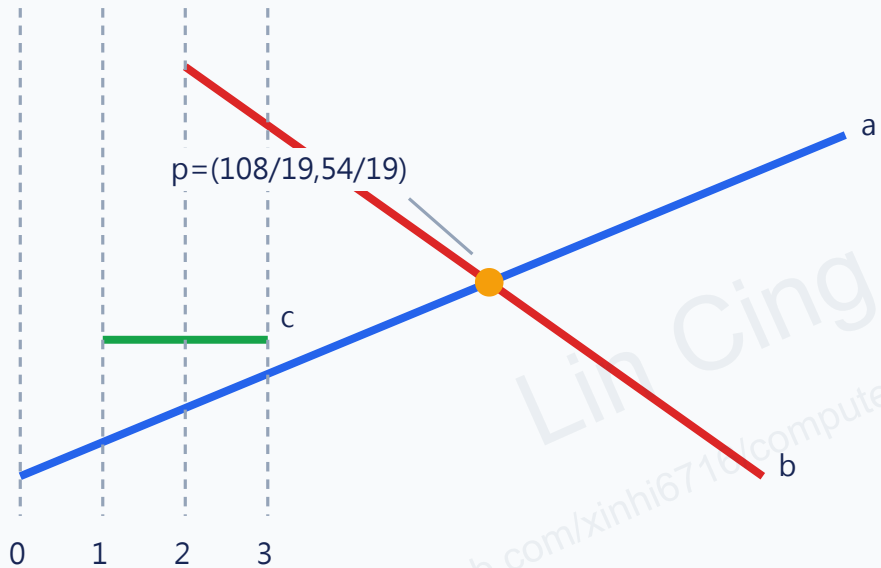
事件	高度與狀態	新測試
x=0 插 a	[a]	無
x=1 插 c	a=1/2,c=2；[a,c]	a,c 不交
x=2 插 b	a=1,c=2,b=6；[a,c,b]	c,b 不交
x=3 刪 c 前	a=3/2,c=2,b=36/7；[a,c,b]	a,b 相交，回傳

a 與水平 y=2 在 x=4 才相遇，超出 c；b 與 y=2 在 x=20/3 才相遇，也超出 c。解 $x/2 = (54 - 6x)/7$ 得 a,b 交點 $(108/19, 54/19)$。刪 c 的事件早於交點。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

用座標逐事件追蹤 （續）

刪除 c 時偵測未來相交



事件 x	T 下→上
0	[a]
1	[a,c]
2	[a,c,b]
3	a,b → true

x=3 < 108/19

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何只看相鄰段就能回答存在性

假設仍未回傳，考慮最左交點 p 的兩段 a,b。p 左側尚無交叉，所以狀態的上下次序不會自行改變。無三段同點時，a,b 在 p 之前或 p 處必成相鄰。

它們首次成相鄰只會由插入其中一段，或刪掉中間最後一段造成。演算法恰在兩種事件檢查完整配對，故應在 p 之前或 p 處回傳。若先在別處回傳，存在性答案也已正確。

平衡樹每事件做常數次 O(log n) 操作，共 2n 個端點，排序 O(n log n)，所以總時間 O(n log n)、空間 O(n)。此論證依賴找到第一對就停止，不能移植成列舉證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同 x 事件不可任意排序

放寬端點 x 互異時，非垂直版本可依 (x,e,y,id) 排序，e=0 是左端點、e=1 是右端點。相同 x 的所有插入先於刪除，再以 y 由低到高；id 僅使同類同點事件可重現。

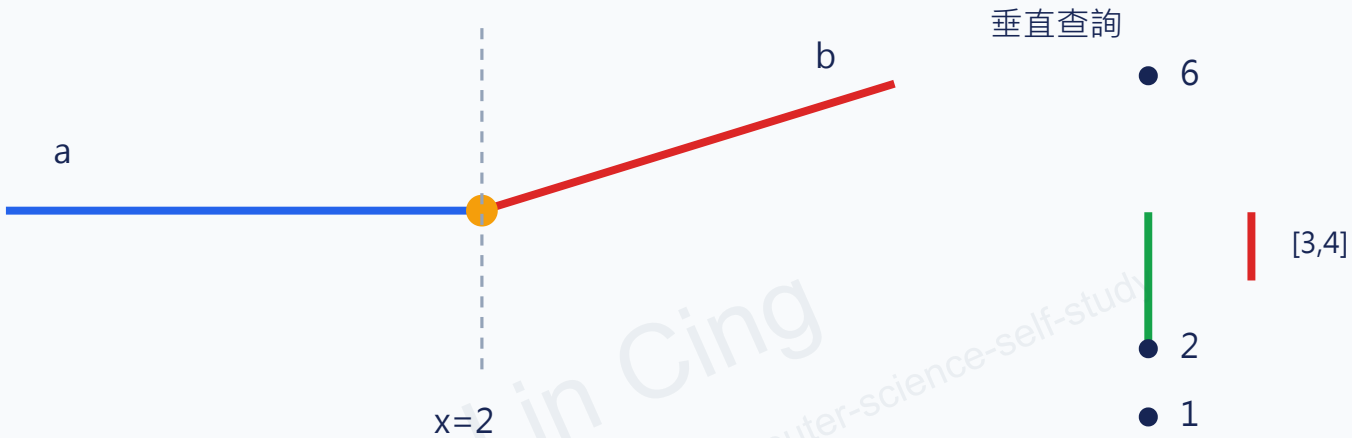
a:(0,1)→(2,1)、b:(2,1)→(4,2) 在共同端點相交。先刪 a 再插 b 會讓兩段從未同在 T。先插 b 則測得到。

同高度可對應真交點，不能用任意 id 取代相交檢查。同點有多段時，需將事件與同高度群組一起處理並驗證邊界，或保留簡化前提；不能只說「隨便打破平手」。CLRS 33.2 的原始證明排除三段同點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同 x 事件不可任意排序 （續）

同 x 的順序與閉區間



先插 b，再刪 a，才能測出共同端點。

[2,4] 命中 y=2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

垂直段是區間查詢

垂直段 v:(x,ylo)→(x,yhi) 不應以單一高度放進一般狀態樹。它可能接觸該區間中的任一非垂直段。

在同 x 批次先插非垂直左端點，接著對垂直閉區間查詢狀態高度，再刪非垂直右端點。存在性可找第一個高度 ≥ylo，若 ≤yhi 即命中。同 x 的垂直段另按 ylo 排序檢查區間重疊；零長度視為單點區間。相同位置的非垂直端點也須檢查。

例 x=2 時活動高度為 1、2、6。v 的 [3,4] 查詢沒有命中，[2,4] 則在邊界 y=2 命中。這描述一般化所需的操作；前頁簡化偽碼未實作完整同點與重疊處理，不能宣稱它已支援所有退化輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

印出後繼續會破壞狀態

a:(0,0)→(4,4)，b:(0,4)→(4,0)，c:(0,5)→(4,5)。在 x<2 時次序為 [a,b,c]，x>2 則為 [b,a,c]。

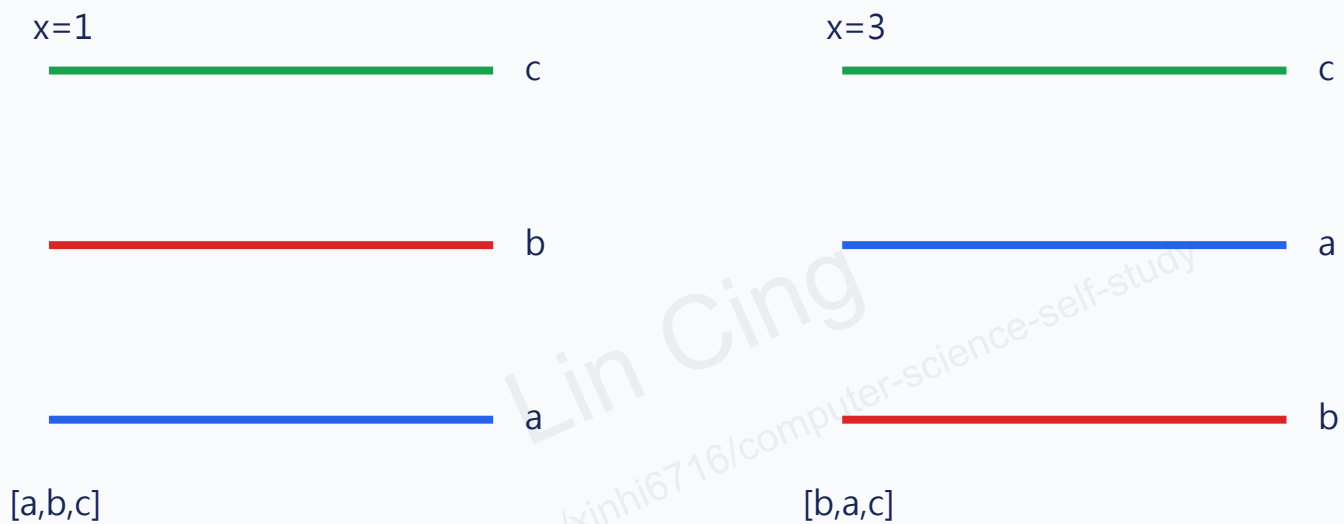
若在插入時發現 a,b 相交只印出再繼續，端點事件之間的樹次序沒有交換，便不再代表幾何鄰居。首次被測出的交點也未必最靠左，因為測試的是整段。

MIT04 的列舉版本另外維護預測交點事件，於交點交換鄰居並更新事件，處理重複與過期事件；在一般位置下為 O((n+k) log n)。共線重疊時須先定義輸出是相交配對、點或重疊段，因為共同點可有無限多個。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

印出後繼續會破壞狀態 （續）

交叉後幾何順序改變



交點事件要交換次序

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

讓判定語義與資料結構一致

層次	必須固定的規格	本節驗證例
方向判定	座標軸、外積順序、算術	行列式 10 與反序 −10
兩段相交	閉段或嚴格交叉、點線段	重疊、端點、延長線反例
掃描存在性	事件次序、狀態鄰居、停止條件	刪 c 才測到 a,b
一般化	同點、垂直、共線的處理	插入後查詢再刪除
列舉	交點事件與輸出定義	交叉後要更新上下次序

練習 A10-P01–P06 分別檢查數值符號、邊界、數值可靠性、逐事件狀態、同座標處理與偵測／列舉差異。方向判定將在凸包的保留或彈出決策中再次使用。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

凸包與最近點對使用不同刪減證據

凸包刪除不能成為極端頂點的點。最近點對刪除不可能改善目前距離的候選配對。兩者都以幾何不變量減少搜尋。固定點集與完整驗算見 data.json 與 check.py。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

明定輸出政策

凸包只輸出極端頂點，逆時針且不重複首點。共線邊上的中間點不輸出；全共線回傳兩端，單點回傳該點，空集合回傳空。凸包先合併相同座標；最近點對保留不同 id 的重複座標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先看固定點集

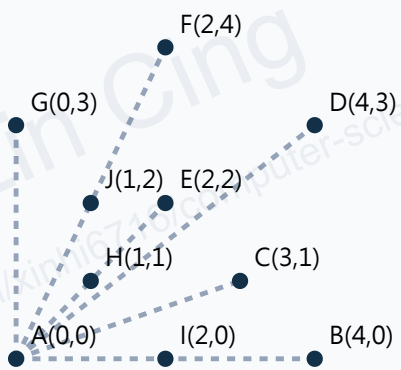
A(0,0), B(4,0), C(3,1), D(4,3), E(2,2), F(2,4), G(0,3), H(1,1), I(2,0), J(1,2)。A 是最低 y 的最左點。orient(a,b,c)=(b−a)×(c−a)，正值為左轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

極角同值必須有政策

繞 A 依極角遞增，未刪除為 I,B,C,D,H,E,J,F,G。同射線只留最遠點，因此移除 I、H、J，保留 B,C,D,E,F,G。J 與 F 也在同一射線。不可只對浮點 atan2 做相等判定，精確版本用叉積與平方距離。

極角排序，A為基點，移除I、H、J



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

掃描規則處理非左轉

令 S 為 stack。依角度讀入 p。只要 $|S|\geq 2$ 且 $\text{orient}(S[-2],S[-1],p)\leq 0$ 就 pop，然後 push p。≤ 0 配合只留極端點；改成 <0 會改變共線輸出，但仍須一起調整角度同值與收尾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每輪 stack 可重播

Point	Popped	Stack bottom to top
B	-	A,B
C	-	A,B,C
D	C	A,B,D
E	-	A,B,D,E
F	E	A,B,D,F
G	-	A,B,D,F,G

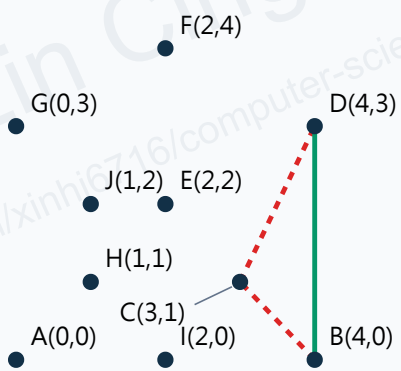
表中起始 stack=[A]，每次列出所有 pop。它與先 push 首三點的標準版本在此非退化資料上等價。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

pop 的局部幾何理由

讀 D 時 orient(B,C,D)=−3，C 被 pop，stack 成 A,B,D。讀 F 時 orient(D,E,F)=−4，E 被 pop，stack 成 A,B,D,F。讀 G 不 pop。被移除點位於 A、先前保留點與新點形成三角形內或邊上，不再是已處理集合的極端點。

讀入D，pop C，保留A、B、D

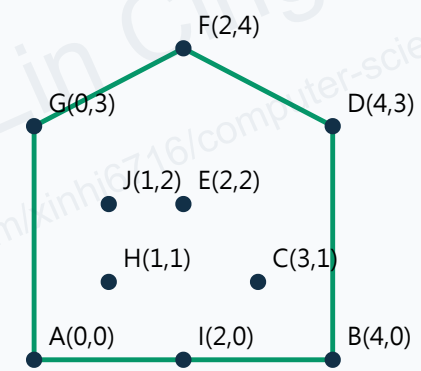


Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正確性不只檢查最後轉向

不變量為處理每一個角度前綴後，stack 正好是該前綴與 A 的凸包極端頂點，依逆時針排列。初始化成立；pop 不改變該前綴凸包，push 新極角的點補上外側邊。終止涵蓋全部點。最後 A,B,D,F,G，全部輸入點位於每條有向邊左側或邊上。

凸包極端頂點 A、B、D、F、G



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

複雜度與 Jarvis march

Graham 排序 O(n log n)，每點至多 push、pop 各一次，掃描 O(n)，空間 O(n)。Jarvis 每輪掃描全部 n 點選下一條支撐邊，h 個極端頂點共 O(nh)。共線候選選最遠點，起點重訪時停止。h 很小有利；h=n 時 $O(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最近點對的完整輸入

固定八點 A(−4,0), B(−3,4), C(−1,1), D(0,5), E(0,1.5), F(2,3), G(4,0), H(5,5)。以平方距離比較，最後才開根號。若不同 id 具有同座標，立即回傳距離 0 的配對。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

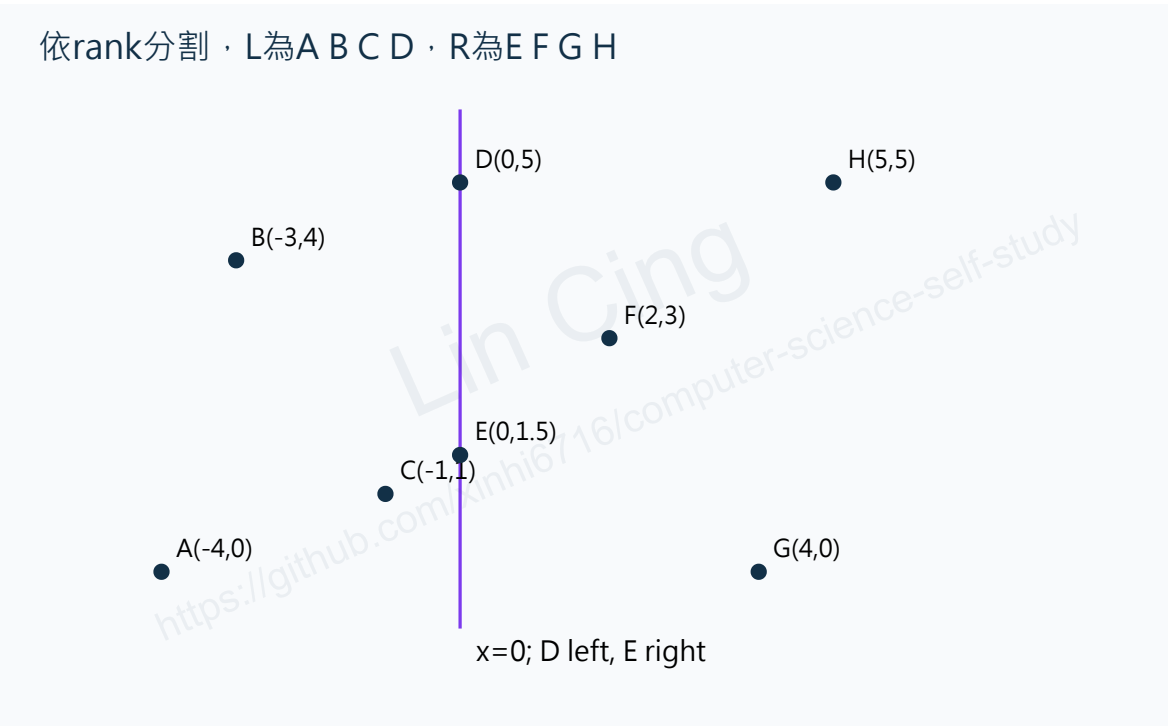
預排序保持兩種順序

只在最外層排序 X=(x,id)、Y=(y,id)。遞迴輸入兩陣列必須具有相同 id 集合。X 依中間索引切成 L 與 R，再以 L 的 id 集合線性穩定過濾 Y。不能每層重新排序 Y。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

重複 x 不用座標判成員

此例 X=A,B,C,D,E,F,G,H，L=A,B,C,D，R=E,F,G,H，分割線 x=0。D、E 的 x 都是 0，依 id/rank 分到不同子問題。若把所有 x≤0 放左邊，子問題可能失衡。



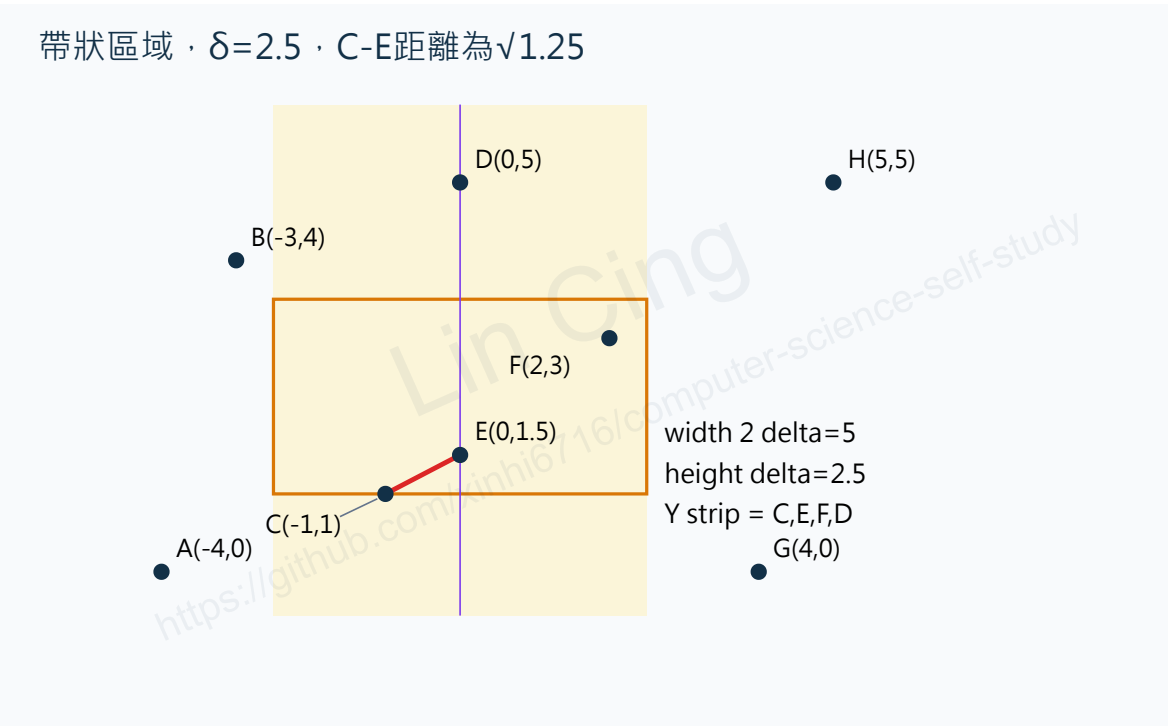
遞迴答案與跨線答案

|P|≤3 時窮舉。左側最小平方距離 10，A-C 與 B-D 同值；右側為 6.25，E-F。因此 δ=min(√10,2.5)=2.5。跨線改善配對的兩點必有 |x−0|<δ 且 |Δy|<δ。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

strip 必須依 y 次序掃描

從 Y 穩定過濾 |x−線|<δ，得到 C(−1,1),E(0,1.5),F(2,3),D(0,5)。C-E 的平方距離為 1.25，距離約 1.118；C-F 為 13，E-F 為 6.25，F-D 為 8。只有 C-E 改善 δ。

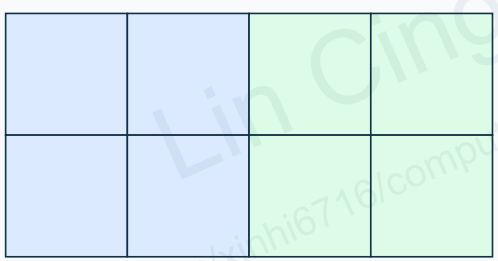


Lin Cing
https://github.com/xinhi6716/computer-science-self-study

常數界來自同側分離

假定 δ>0 且遞迴已證明 L 內、R 內任意兩點距離≥δ。對 strip 中 p，只考慮其上方 y 差小於 δ 的點，落在寬 2δ、高 δ 矩形。左右各分四個 δ/2×δ/2 格，同一側每格最多一點，因格對角線 δ/√2<δ。線上點仍按 id 所屬側計數。

每側四格，每格邊長δ/2



2 delta by delta; cell diagonal = delta/sqrt(2)
At most 4 left + 4 right = 8 including p

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何最多看後七個

每側最多 4 點，矩形合計最多 8 點，包含 p，因此後方候選最多 7 個。Y 必須有一致的 y、id 次序；可用半開格規則唯一分配邊界。此界不假設左右跨側點已互相分離。δ=0 先結束，不能再套正方形裝填。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴式取決於資料流

預排序 $O(n \log n)$，每層分割 X、過濾 Y、建 strip、每點常數比較總共 $O(n)$。$T(n)=2T(n/2)+O(n)=O(n \log n)$。若每層排序 strip，變成 $O(n \log^2 n)$。深度 $O(\log n)$，循序遞迴釋放陣列可用 $O(n)$ 額外空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

來源與獨立驗算

CLRS 第三版 §33.3–33.4，印刷頁 1029–1043，PDF 頁 1050–1064。原掃描 note PDF 頁46–47支持凸包與Graham；沒有從這兩頁推定最近點對內容。MIT 2004 L18 PDF頁3–11支持分治與裝填；2005本地21份清單無計算幾何講次。check.py窮舉固定點集全部45/28個配對並驗證支撐邊、stack與分治答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 33. MIT 6.046J Fall 2004 computational geometry lecture slides, segment intersection and closest pair. Original handwritten algorithms notes (演算法_wjungle), computational geometry pages.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study